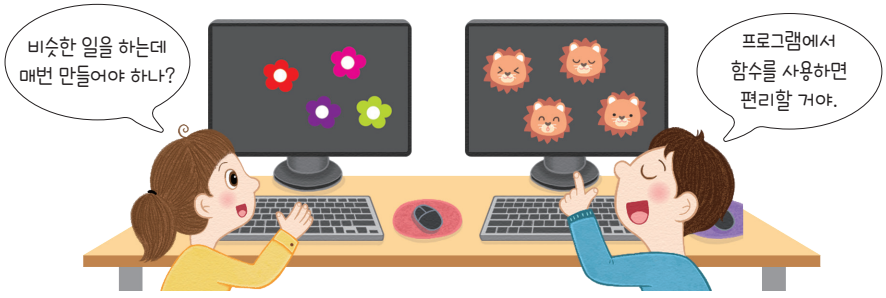
1 함수

프로그램에서 함수(function)는 특정 기능을 수행하기 위해 여러 개의 명령문을 작은 단위의 코드로 묶은 것을 말한다. 프로그램에서 같은 작업을 여러 번 반복하여 사용하는 부분을 함수로 만들고, 필요할 때마다 이 함수를 호출하여 사용하면 프로그램을 효율적으로 작성하고 관리할 수 있다.



파이썬에서 함수는 내장 함수, 외장 함수, 사용자 정의 함수 등이 있다. 내장 함수는 파이썬을 설치할 때 함께 제공되며, 외장 함수는 파이썬 라이브러리 모듈에 포함되어 있다. 그리고 사용자 정의 함수는 사용자가 필요할 때마다 직접 만들어서 사용한다.

내장 함수 종류 확인하기
인터넷 검색창에서 'python built-in functions'으로 검색하여 내장 함수의 종류를 확인할 수 있다.

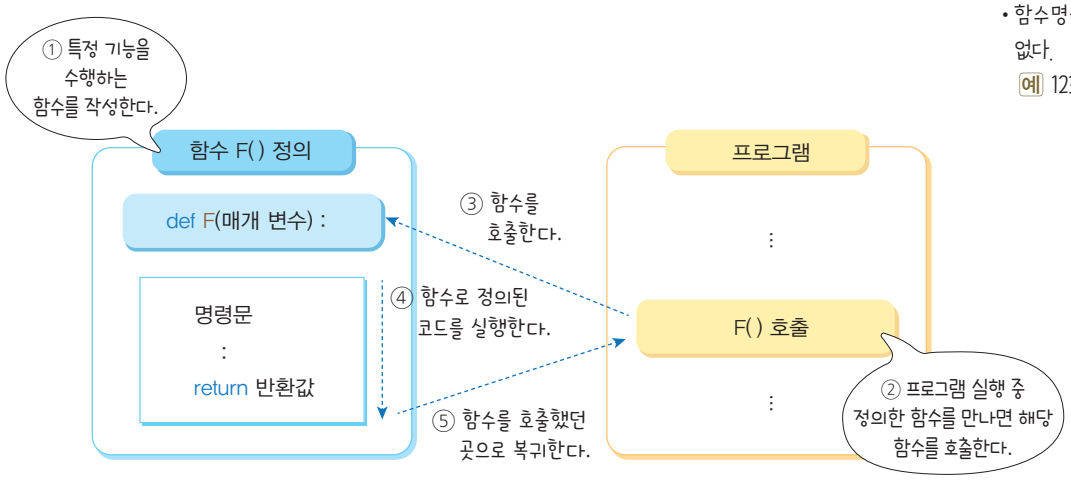
외장 함수 사용하기
모듈을 import하여 외장 함수를 사용할 수 있다.

예 math 모듈의 gcd 외장 함수 사용하기

```
import math      # math 모듈
math.gcd(3,18) # gcd 외장 함수
```

1 사용자 정의 함수

사용자가 원하는 기능을 직접 정의하는 함수이다. 함수를 사용하려면 특정 기능을 수행하는 함수를 미리 만들어 놓아야 한다. 이것을 '함수 정의'라 하고, 작업이 필요할 때 정의된 함수를 호출해서 실행시키는 것을 '함수 호출'이라고 한다. 함수를 이용한 프로그램의 실행 과정은 다음과 같다.



▲ 함수의 실행 과정

탐색 함수명 작성 시 주의 사항을 읽고, 제시한 예가 맞는지 O, X를 적어 보자.

- 숫자, 한글, 알파벳, 언더바(_)를 사용할 수 있다.
  예 turn_left( )
- 공백을 사용할 수 없다.
  예 turn right( )
- 키워드는 사용할 수 없다.
  예 if( ), and( )
- 함수명을 숫자로 시작할 수 없다.
  예 123abcd( )

함수 정의와 함수 호출 (PDF)

매개 변수, 인수, 반환값
- 매개 변수: 함수 호출 시 전달된 값을 받는 변수이다.
- 인수: 함수 호출 시 보내는 값이다.
- 반환값: 함수의 결과를 호출한 곳으로 전달하는 값으로 return문을 사용한다.
- 매개 변수, 인수, 반환값은 필수 요건은 아니다.

return
함수를 호출한 위치로 특정 값을 전달하고 함수 실행을 종료한다.

2 다양한 사용자 정의 함수 만들기

사용자 정의 함수는 함수를 호출할 때 인수가 있거나 없을 수 있고, 함수 정의 내의 반환값 또한 있거나 없을 수 있다.

방법 1 인수와 반환값이 없는 경우

```
01 def f() :                    # 함수 정의
02     print("Hello world")
03
04 f()                          # 함수 호출
```

실행 결과
Hello world

04행에서 인수 없이 함수 f( )를 호출하면, 02행에서 "Hello World"를 출력한 다음 함수를 종료하고 04행으로 복귀하여 프로그램을 종료한다.

방법 2 인수가 없고 반환값이 있는 경우

```
01 def f() :                    # 함수 정의
02     a = "Hello world"
03     return a                 # a값 반환하기
04
05 b = f()                      # 함수 호출: 함수 f( )의 반환값을 b에 저장하기
06 print(b)
```

실행 결과
Hello world

05행에서 인수 없이 함수 f( )를 호출하면, 02행에서 문자열 "Hello World"를 변수 a에 저장한 후 03행에서 a의 값을 가지고 05행으로 복귀하여 이 값을 변수 b에 저장한다. 이후 6행에서 b의 값, 즉 "Hello World"를 출력하고 프로그램을 종료한다.

활동 21

개발

[실행 결과]와 같이 채팅방에 입장할 때 나타나는 환영 인사를 함수 g( )로 구현해 보자.

```
01 def g(name) :                # 함수 정의하기
02 [                            ]
03
04
05
06
07                              # 함수 호출하기
08 g("위풍당당 사자")
09 g("행복한 나무늘보")
10 g("뒤뚱뒤뚱 아기 오리")
```

실행 결과
환영합니다.
위풍당당 사자 님 입장
공지 사항을 꼭 확인하세요.

환영합니다.
행복한 나무늘보 님 입장
공지 사항을 꼭 확인하세요.

환영합니다.
뒤뚱뒤뚱 아기 오리 님 입장
공지 사항을 꼭 확인하세요.

방법 3 인수가 있고 반환값이 없는 경우

```
01 def f(a) :          # 함수 정의: 매개 변수 a = 3
02     print(a)
03
04 f(3)                # 함수 호출: 인수는 3
```

실행 결과

```
3
```

04행에서 함수 f(3)를 호출하면 01행에서 매개 변수 a에 3을 저장한 후 02행에서 a의 값, 즉 3을 출력한 다음 04행으로 복귀한 후 프로그램을 종료한다.

방법 4 인수와 반환값이 있는 경우

```
01 def f(a) :          # 함수 정의: 매개 변수 a = 3
02     x = a + 3
03     return x        # x의 반환값 전달
04
05 c = 3
06 b = f(c)            # 함수 호출: 함수 f( )의 인수는 c = (3)
07 print(b)
```

실행 결과

```
6
```

05행에서 변수 c에 3을 저장하고 06행에서 함수 f(c)를 호출하면, 01행에서 매개 변수 a가 3을 저장한 후 02행에서 a와 x의 값을 더해 변수 x에 저장한다. 03행에서 이 값을 변수 x에 저장한 다음 06행으로 복귀한다. 이때 함수에서 반환된 값은 b에 저장되고 07행에서 b의 값을 출력한 다음 프로그램을 종료한다.

math 모듈과 관련 함수

- 수학과 관련된 함수 및 상수를 제공하는 모듈이다.
- math.ceil( ): 주어진 숫자를 올림하여 정수를 반환한다.
- math.floor( ): 주어진 숫자를 내림하여 정수를 반환한다.

```
01 import math
02 math.ceil(3.14)   # 4 반환
03 math.floor(3.56)  # 3 반환
```

활동 22

개발

놀이기구의 탑승 가능 인원(capacity)과 대기 인원(waiting)을 입력했을 때 몇 번 만에 놀이기구를 탑승할 수 있는지 함수로 정의해 보자.

```
01 import math
02 def f(a, b) :                          # 함수 정의
03     if [                    ]          # 탑승 가능 인원이 대기 인원보다 크거나 같으면 1
04         return 1
05     else :
06         return math.ceil([          ])  # 탑승 가능 인원이 대기 인원보다 작으면
07                                        # (대기 인원 / 탑승 가능 인원) 후 소수점 올림하기
08 capacity = int(input("탑승 가능 인원: "))
09 waiting = int(input("대기 인원: "))
10 print(f(capacity, waiting))
```

실행 결과

```
탑승 가능 인원: 12 Enter↵
대기 인원: 33 Enter↵
3
```

3 지역 변수와 전역 변수

프로그램에서 변수가 선언된 위치에 따라 변수를 사용할 수 있는 범위가 달라진다. 변수는 함수와 같은 특정 지역에서만 사용하는 지역 변수(local variable)와 함수 외부에 생성되어 프로그램 전체에서 사용되는 전역 변수(global variable)가 있다.

함수 내부에서 전역 변숫값 변경하기

함수 내부에서 다음의 형식으로 선언하여 바꿀 수 있다.

▶ 형식

global 전역 변수

```
count = 0
def add(a, b) :
    global count
    c = a + b
    print(a,"+", b,"=", c)
    count += 1

add(3, 5)
add(10, 14)
print("덧셈 횟수", count)
```

실행 결과

```
3 + 5 = 8
10 + 14 = 24
덧셈 횟수 2
```

예제 35 지역 변수 count와 전역 변수 count의 값을 출력하는 프로그램을 작성해 보자.

```
count = 20                # 전역 변수
def func() :
    count = 10            # 지역 변수
    print("함수 내부 변수: ", count)

func()
print("함수 외부 변수: ", count)
```

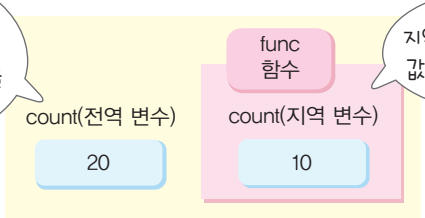
전역 변수의 사용 영역

지역 변수의 사용 영역

실행 결과

```
함수 내부 변수: 10
함수 외부 변수: 20
```

전역 변수는 프로그램 전역에서 값을 변경할 수 있지만, 함수 내부에서 값을 변경할 수 없어요.



지역 변수는 함수 내부에서만 값을 읽고 변경할 수 있어요.

활동 23 탐색 개발

순차 탐색을 함수로 정의하여 탐색 위치(result), 탐색 횟수(count)를 출력해 보자. Link 103쪽

```
count = 0                          # 탐색 횟수 초기화
def L_search(a, target) :          # 순차 탐색 함수 정의
  global count                     # 전역 변수 count 사용
  n = len(a)                       # a 개수 저장하기
  for i in range(n) :              # i를 0부터 n-1까지 반복하기
    count += 1                     # 탐색 횟수 증가하기
    if a[i] == target :            # a[i]가 target과 같은가?
      return i                     # 같으면 위치 [i]를 반환하고 함수 종료하기
  return -1                        # traget이 a에 없으면 -1을 반환하기

a = [3, 5, 7, 8, 9, 10, 14]        # a 리스트 생성하기
target = 14                        # 찾을 값 설정하기
result = L_search(a, target)       # 순차 탐색 함수 호출
print("탐색 위치: ", result)
print("탐색 횟수: ", count)
```

실행 결과

```
탐색 위치: 6
탐색 횟수: 7
```

활동 24

탐색 개발

이진 탐색을 살펴보고 활동 23의 순차 탐색 방법과 비교해 보자. Link 104쪽

1 다음은 이진 탐색의 처리 과정이다. 탐색값(target)이 14일 때, 왼쪽값, 중앙값, 오른쪽값을 찾아 각각 left, mid, right로 적어 보자.

초기 상태	인덱스	0	1	2	3	4	5	6
	값	3	5	7	8	9	10	14
단계 1	인덱스	0	1	2	3	4	5	6
	값	3	5	7	8	9	10	14
		left			mid			right
단계 2	인덱스	0	1	2	3	4	5	6
	값	3	5	7	8	9	10	14
						left	mid	right
단계 3	인덱스	0	1	2	3	4	5	6
	값	3	5	7	8	9	10	14
								mid

2 이진 탐색을 함수로 정의하여 탐색 횟수(count)를 출력해 보자.

프로그램 맛보기 (코드)

```
count = 0                          # 탐색 횟수 초기화
def B_search(a, target) :          # 이진 탐색 함수 정의
  global count                     # 전역 변수 count 사용
  left, right = 0, len(a) - 1      # 왼쪽 끝 인덱스와 오른쪽 끝 인덱스 설정하기
  while left <= right :            # 왼쪽 끝 인덱스가 오른쪽 끝 인덱스를 지나치면 중지하기
    mid = (left + right) // 2      # 중간 인덱스 설정하기
    count += 1                     # 탐색 횟수 증가하기
    print("left-", left, "mid-", mid, "right-", right) # 왼쪽-중간-오른쪽 인덱스 출력하기
    if a[mid] == target :          # 중간값이 target과 같으면 인덱스 반환하기
      return mid
    elif a[mid] > target :         # 중간값이 target보다 크면 왼쪽 부분 탐색하기
      right = mid - 1
    else :                         # 중간값이 target보다 작으면 오른쪽 부분 탐색하기
      left = mid + 1
  return -1                        # target을 찾지 못한 경우 -1 반환하고 함수 종료하기

a = [7, 14, 3, 9, 8, 5, 10]        # 탐색할 리스트 저장하기
a.sort()                           # a 리스트를 오름차순으로 정렬하기
target = 14                        # 탐색값을 target에 저장하기
result = B_search(a, target)       # B_search( ) 함수 호출하기
print("탐색 위치: ", result)
print("탐색 횟수: ", count)
```

이진 탐색은 사전에 데이터가 정렬되어야 해요.

실행 결과

```
left- 0 mid- 3 right- 6
left- 4 mid- 5 right- 6
left- 6 mid- 6 right- 6
탐색 위치: 6
탐색 횟수: 3
```